



Architecture Blueprint

Full-Stack International Dating App

Olusegun Abayomi Stephen · v1.0 · 2024

Node.js	React 18	Socket.io	SQLite	WebRTC	Claude AI
JWT Auth	REST API	Real-Time	Geolocation	Multi-Currency	Admin Dashboard

Table of Contents

1		
.	System Overview	2
2		
.	Technology Stack	2
3		
.	Directory Structure	3
4		
.	Backend Architecture	3
5		
.	Frontend Architecture	5
6		
.	Real-Time Layer (Socket.io)	6
7		
.	WebRTC — Audio & Video Calls	7
8		
.	Database Schema	8

9		
.	Authentication & Security	9
1		
0		
.	Geolocation & Distance Engine	9
1		
1		
.	AI Integration	10
1		
2		
.	Subscription & Monetisation	10
1		
3		
.	Admin Dashboard	11
1		
4		
.	API Reference	11
1		
5		
.	Data Flow Diagrams	12

1. System Overview

MyLover is a production-ready, full-stack international dating application. It is built on a client-server architecture where a **Node.js/Express** backend serves a **React 18** single-page application. Real-time communication is handled by **Socket.io** over WebSockets, and peer-to-peer audio/video calls are implemented using the browser-native **WebRTC** API with Google STUN servers for NAT traversal. **Claude AI (Sonnet 4.6)** powers the dating coach, icebreaker generator, and automated content moderation.

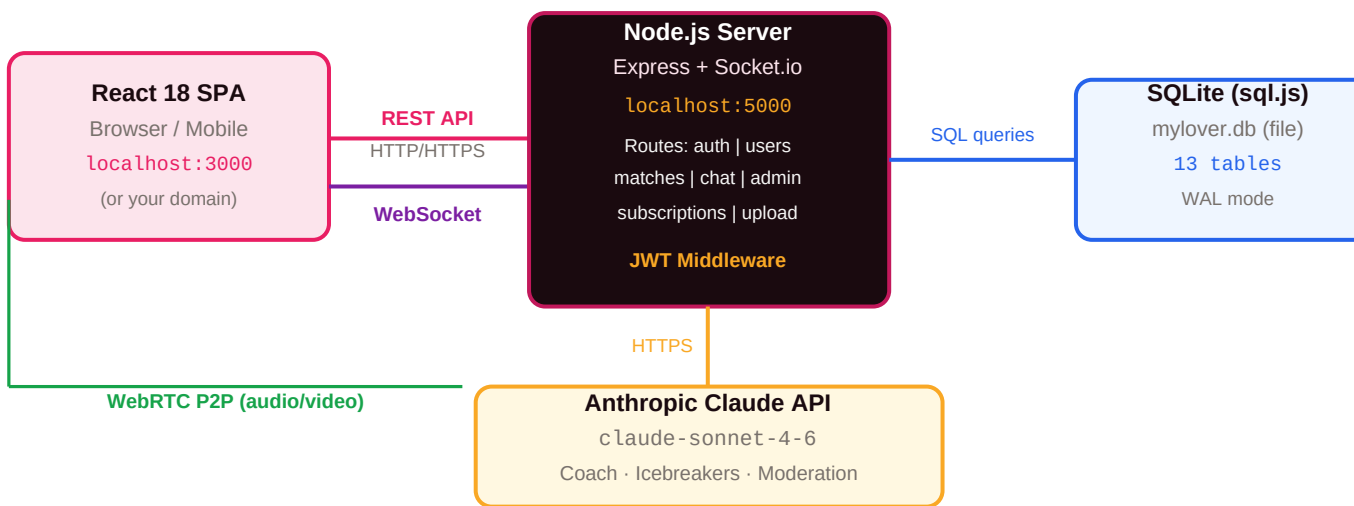


Figure 1 — High-level system architecture

2. Technology Stack

Layer	Technology	Version	Purpose
Runtime	Node.js	v22+	Server-side JavaScript runtime
Web Framework	Express.js	4.18	HTTP routing, middleware, REST API
Real-Time	Socket.io	4.7	WebSocket events — chat, likes, calls
Database	sql.js (SQLite)	1.10	Embedded SQL database, file-based
Frontend	React	18.2	Component-based SPA
Routing	React Router	6.21	Client-side page navigation
Auth	JWT + bcryptjs	9.0 / 2.4	Token auth + password hashing
Calls	WebRTC (native)	Browser	Peer-to-peer audio & video
STUN	Google STUN	Public	NAT traversal for WebRTC
AI	Anthropic Claude	Sonnet 4.6	Dating coach, moderation, icebreakers
File Upload	Multer	1.4	Profile photo & media uploads

Geo	Haversine (custom)	—	Distance calculation in metres
Security	Helmet + rate-limit	7.1 / 7.1	HTTP headers + brute-force protection
Package Mgr	npm	10+	Dependency management

3. Directory Structure

The project is a Node.js monorepo. The **server/** directory contains the full Express/Socket.io backend. The **client/** directory is a standard Create-React-App project. Packages are installed at the root level and shared between both halves.

```
mylover/
├── package.json          ← root (shared node_modules)
├── start.sh              ← one-command dev launcher
├── README.md
├──
├── server/
│   ├── index.js          ← entry point, Socket.io, async startup
│   ├── db.js             ← schema creation, admin seeding
│   ├── db_sqlite_shim.js ← synchronous-API wrapper for sql.js
│   ├── package.json
│   ├── .env.example
│   ├── middleware/
│   │   ├── auth.js       ← JWT verify (HTTP + Socket)
│   │   └── routes/
│   │       ├── auth.js   ← register, login, /me, trial-status
│   │       ├── users.js  ← profile, discover, distance, block, report
│   │       ├── matches.js ← matches list, likes-received
│   │       ├── chat.js   ← conversations, messages, AI coach
│   │       ├── subscriptions.js ← plans, subscribe, cancel
│   │       ├── admin.js  ← full admin API (stats, ban, AI scan...)
│   │       └── upload.js ← photo upload (multer)
│   ├── uploads/          ← user photos (auto-created)
│   ├── data/
│   │   └── mylover.db    ← SQLite file (auto-created)
│   └──
├── client/
│   ├── package.json
│   ├── src/
│   │   ├── App.js        ← router + route guards
│   │   ├── index.css     ← full design system (CSS variables)
│   │   ├── index.js      ← React root
│   │   ├── context/
│   │   │   ├── AuthContext.js ← global auth state (user, token, trial)
│   │   │   └── SocketContext.js ← persistent socket connection
│   │   ├── utils/
│   │   │   ├── api.js     ← all fetch() wrappers
│   │   │   └── geo.js     ← Haversine, browser geolocation
│   │   ├── components/
│   │   │   └── AppShell.js ← top nav + bottom mobile nav
│   │   └── pages/
│   │       ├── LandingPage.js ← public homepage
│   │       ├── LoginPage.js
│   │       ├── RegisterPage.js
│   │       ├── SetupProfilePage.js ← 4-step onboarding
│   │       ├── DiscoverPage.js ← swipe cards, geo filter, match anim
│   │       ├── MatchesPage.js ← match list + likes received
│   │       └── ChatPage.js ← real-time chat + WebRTC calls + AI
```

```
■■■ ProfilePage.js    ← edit profile, photo upload
■■■ SubscribePage.js ← multi-currency premium plans
■■■ AdminPage.js     ← full admin dashboard
```

4. Backend Architecture

The backend is a single Node.js process handling both HTTP and WebSocket traffic on the same port. It initialises asynchronously: the sql.js WASM engine is loaded first, the database schema is created, and then Express routes and Socket.io event handlers are registered.

4.1 Request Lifecycle

1	Client sends HTTP request Browser → Express server:5000
2	Rate limiter checks express-rate-limit (300 req/15 min general, 30 for /auth)
3	CORS validation helmet + cors middleware verify origin header
4	JWT authentication middleware/auth.js decodes Bearer token, loads user from DB
5	Route handler executes Queries SQLite via db_sqlite_shim, returns JSON
6	Response sent JSON response with appropriate HTTP status code

4.2 API Routes Summary

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Create account
POST	/api/auth/login	No	Login, returns JWT
GET	/api/auth/me	JWT	Current user object
GET	/api/auth/trial-status	JWT	Free trial days left
PUT	/api/users/profile	JWT	Update profile fields + lat/Ing
GET	/api/users/discover	JWT	Paginated discovery feed with distance
GET	/api/users/:id	JWT	Single profile with distance
POST	/api/users/:id/block	JWT	Block a user
POST	/api/users/:id/report	JWT	Report a user to admin
GET	/api/matches	JWT	All mutual matches + conversations
GET	/api/matches/likes-received	JWT+Premium	Who liked me
GET	/api/chat/conversations	JWT	Conversation list with unread count

GET	/api/chat/conversations/:id/messages	JWT	Paginated message history
POST	/api/chat/ai-coach	JWT	AI dating coach (Claude)
POST	/api/chat/ai-icebreaker	JWT	Generate opening messages
GET	/api/subscriptions/plans	No	Plan list in chosen currency
POST	/api/subscriptions/subscribe	JWT	Activate plan by payment ref
POST	/api/upload/profile-photo	JWT	Upload main profile photo
POST	/api/upload/photos	JWT	Upload up to 6 gallery photos
GET	/api/admin/stats	Admin	Dashboard statistics
GET	/api/admin/users	Admin	Paginated user list
POST	/api/admin/users/:id/ban	Admin	Ban user
GET	/api/admin/messages	Admin	All messages with search
POST	/api/admin/messages/ai-scan	Admin	AI content moderation scan
GET	/api/admin/reports	Admin	Pending reports queue

5. Frontend Architecture

The frontend is a React 18 Single-Page Application using React Router v6 for client-side navigation. Global state (auth, socket connection) is managed via Context API. All server communication goes through `utils/api.js`, a thin fetch wrapper that automatically attaches the JWT Bearer token.

Page / Component	Route	Key Responsibilities
LandingPage	/	Public homepage, feature showcase, CTA buttons
LoginPage	/login	Email/password form, JWT storage
RegisterPage	/register	Account creation, gender, DOB, country
SetupProfilePage	/setup	4-step onboarding: photo, bio, interests, location
DiscoverPage	/discover	Swipe cards, touch gestures, distance display, match animation, geo filters
MatchesPage	/matches	Match grid, likes-received tab, like-back button, socket notifications
ChatPage	/chat/:id	Real-time messaging, typing indicators, read receipts, WebRTC calls, AI coach, ic
ProfilePage	/profile	Edit profile, photo upload, trial status, AI credits
SubscribePage	/subscribe	Multi-currency plan selector, payment reference flow
AdminPage	/admin/*	Full admin dashboard — stats, users, messages, reports, logs
AppShell	(wrapper)	Top navbar (desktop) + bottom tab bar (mobile), online status
AuthContext	(context)	user, login(), register(), logout(), isPremium, trialStatus
SocketContext	(context)	Persistent socket, emit(), on(), onlineUsers Set

5.1 State & Data Flow

User actions in the browser follow a consistent pattern: **(1)** UI event triggers an API call or socket emit, **(2)** the server processes and responds, **(3)** React state is updated, **(4)** the component re-renders. Socket events (`new_message`, `its_a_match`, `new_like`) are subscribed to via the `on()` helper in `SocketContext` and automatically unsubscribed on component unmount via the returned cleanup function.

6. Real-Time Layer — Socket.io Events

Event Name	Direction	Payload	Purpose
join_conversation	Client → Server	{ conversationId }	Subscribe to a chat room
send_message	Client → Server	{ conversationId, content, messageType }	Send a text message
new_message	Server → Client	{ id, conversationId, senderId, content }	Deliver a message to room
message_blocked	Server → Client	{ reason, upgradeRequired }	Free tier limit reached
typing	Client → Server	{ conversationId, isTyping }	Typing start/stop
user_typing	Server → Client	{ userId, isTyping }	Forward typing state
mark_read	Client → Server	{ conversationId }	Mark messages as read
messages_read	Server → Client	{ conversationId, readBy }	Confirm read status
send_like	Client → Server	{ targetUserId, isSuper }	Like or super-like a profile
new_like	Server → Client	{ fromUserId, isSuper }	Notify target of incoming like
its_a_match	Server → Client	{ matchId, conversationId, userId }	Notify like detected
user_online	Server → Broadcast	{ userId }	User connected
user_offline	Server → Broadcast	{ userId }	User disconnected
call_offer	Client → Server	{ targetUserId, offer, conversationId }	Initiate WebRTC call
incoming_call	Server → Client	{ fromUserId, offer, conversationId }	Notify callee
call_answer	Client → Server	{ targetUserId, answer }	Accept call, send SDP answer
call_answered	Server → Client	{ answer }	Forward answer to caller
ice_candidate	Bidirectional	{ targetUserId, candidate }	ICE candidate exchange
end_call	Client → Server	{ targetUserId }	Hang up / decline call
call_ended	Server → Client	{ }	Notify the other party

7. WebRTC — Audio & Video Calls

Audio and video calls are implemented using the browser-native WebRTC API. The Socket.io server acts purely as a **signalling channel** — it never carries media. Audio and video streams travel directly between the two browsers over a peer-to-peer connection.

1	Caller	Caller clicks ■ or ■
2	Caller	getUserMedia() — get mic/camera
3	Caller	createOffer() — generate SDP
4	Signalling	socket.emit(call_offer, offer)
5	Signalling	Server → socket.emit(incoming_call)
6	Callee	Callee sees Accept/Decline popup
7	Callee	Callee accepts → getUserMedia()
8	Callee	createAnswer() — generate SDP
9	Signalling	socket.emit(call_answer, answer)
10	Signalling	Server → socket.emit(call_answered)
11	P2P Setup	ICE candidates exchanged (bidirectional)
12	STUN	STUN (stun.l.google.com) resolves public IPs
13	Live Call	P2P connection established — media flows!

STUN Server Used: stun:stun.l.google.com:19302 (free, public)

For production behind strict NAT (corporate networks, some mobile carriers): add a TURN server (e.g. Twilio Network Traversal Service, Xirsys, or self-hosted coturn).

8. Database Schema

users

Core user record. Stores all profile fields, lat/lng for geolocation, is_admin, is_premium, is_banned, ai_credits, free-trial timestamp.

subscriptions

One row per subscription purchase. Tracks plan, currency, price, payment reference, start and expiry dates.

likes

Each like event: from_user_id → to_user_id. UNIQUE constraint prevents duplicates. is_super flag for super-likes.

matches

Created when likes are mutual. user1_id/user2_id stored in alphabetical order (smaller UUID first) to ensure uniqueness.

conversations

One per matched pair. Mirrors the matches table but also caches last_message and last_message_at for the inbox list.

messages

Each chat message. Linked to a conversation. is_flagged and flag_reason support the admin moderation flow.

reports

User abuse reports submitted via the Report button. status transitions: pending → resolved.

blocks

Blocked user pairs. Blocked users are excluded from the discovery feed automatically.

notifications

In-app notification queue (likes, matches, system messages). Read/unread tracking.

admin_logs

Immutable audit trail of every admin action (ban, verify, grant premium, flag message).

Persistence: The sql.js database is held in memory and written to disk (mylover.db) every 2 seconds when dirty, and immediately after every DDL statement. For production scale (>10,000 users), replace with **PostgreSQL** — all queries use the same prepare().get()/all()/run() API so the change is minimal.

9. Authentication & Security

JWT Tokens	JSON Web Tokens signed with HS256. 30-day expiry. Stored in localStorage on the client.
Password Hashing	bcryptjs with salt rounds = 12. Passwords are never stored in plaintext.
Rate Limiting	express-rate-limit: 300 requests per 15 minutes globally, 30 for /api/auth/* to prevent brute force.
Helmet.js	Sets secure HTTP headers: X-Frame-Options, X-Content-Type-Options, HSTS, etc.
CORS	Configured to allow only the CLIENT_URL origin (localhost:3000 in dev, your domain in production).
Socket Auth	Every Socket.io connection verifies the JWT from handshake.auth.token before allowing any events.
Admin Guard	requireAdmin middleware checks is_admin = 1 on the user record for all /api/admin/* routes.
Input Validation	Route handlers validate required fields; SQL parameterised queries prevent injection.
File Upload	Multer restricts uploads to image MIME types only, max 10MB per file.
Ban System	Banned users receive 403 on every authenticated request; socket connections are rejected.

10. Geolocation & Distance Engine

Distance between users is calculated server-side using the **Haversine formula**, which gives the great-circle distance between two points on Earth given their latitude and longitude.

```
haversine(lat1, lng1, lat2, lng2) = 2 × R × arctan2( sqrt(a), sqrt(1-a) )  
where a = sin2(Δlat/2) + cos(lat1) × cos(lat2) × sin2(Δlng/2) and R =  
6,371,000 m
```

- 1 User opens Discover → browser requests location permission (navigator.geolocation)
- 2 On grant: lat/lng sent to PUT /api/users/profile → saved to users.lat, users.lng
- 3 GET /api/users/discover fetches current user lat/lng from DB
- 4 For every candidate profile: haversine(myLat, myLng, theirLat, theirLng) computed in-process

5	Profiles with location are sorted nearest-first; those without location appear at the end
6	distanceLabel field returned: "345 m away", "2.3 km away", "127 km away"
7	Optional maxDistance filter (km): profiles beyond threshold are excluded before pagination

11. AI Integration (Claude Sonnet 4.6)

Dating Coach POST /api/chat/ai-coach

Personalised conversational coach. Knows user name, country, gender, bio. Gives dating advice, message feedback, relationship guidance. Costs 1 AI credit per exchange.

Icebreaker Generator POST /api/chat/ai-icebreaker

Given both profiles (interests, bio, country), generates 3 creative, culturally-aware opening messages. Costs 1 AI credit.

Content Moderation Scan POST /api/admin/messages/ai-scan

Admin-triggered batch scan. Sends up to 100 messages to Claude, which returns a JSON list of flagged message IDs with reasons (scam, harassment, explicit content, etc.).

AI Credits: Free users receive 100 credits on registration. Premium users receive an additional 500 credits on each subscription activation. Credits are deducted atomically in the DB and refunded if the API call fails.

12. Subscription & Monetisation

The app uses a **freemium model** with a 60-day full-access free trial for new users. After the trial, free accounts are limited to 5 messages per day and cannot see who liked them or make calls. Three paid plans are available in five currencies.

Plan	Duration	USD	NGN	GBP	EUR	KES
Weekly	7 days	\$4.99	₦7,999	£3.99	€4.49	KSh 699
Monthly	30 days	\$14.99	₦22,999	£11.99	€13.99	KSh 1,999
Yearly	365 days	\$99.99	₦149,999	£79.99	€89.99	KSh 12,999

Monthly plan marked as "Most Popular" in the UI. Yearly saves ~44% vs monthly.

Payment Flow (current): Manual payment reference — user transfers money to the configured bank account, enters the transaction reference, and the subscription activates immediately. **Recommended upgrades:** Paystack (Nigeria), Stripe (international), Flutterwave (Africa). Webhook handlers can be added to server/routes/subscriptions.js with minimal changes.

13. Admin Dashboard

Dashboard

Real-time stats: total users, active today, new this week/month, premium count, total messages, flagged messages, pending reports, active subscriptions. Top countries by user count. Subscription breakdown by plan.

User Management

Full paginated user table. Search by name or email. Filter by banned/premium/verified. Actions per user: Verify (blue tick), Grant Premium, Ban (with reason), Unban. All actions logged to admin_logs.

Message Monitor

Read all conversation messages across the platform. Search by content. Filter to flagged-only. Manually flag or unflag any message. One-click AI Scan: sends a batch to Claude for automatic moderation.

Reports Queue

View all pending user reports. See reporter, reported user, reason, and details. Actions: Ban the reported user directly, or Resolve with a recorded action. Resolved reports archived with timestamp and reviewer.

Subscriptions

Full payment history — user, plan, price, currency, status, expiry, payment reference. Useful for manual verification and revenue tracking.

Audit Logs

Immutable log of every admin action: who did what, to which user, when, and why. Provides accountability and a paper trail for moderation decisions.

14. Error Codes

Code	Meaning	Common Cause
200	OK	Request succeeded
201	Created	New resource created (register, message)
400	Bad Request	Missing required field
401	Unauthorised	No token, expired token, wrong password

403	Forbidden	Banned account, insufficient plan, not admin
404	Not Found	User or resource does not exist
409	Conflict	Email already registered
500	Server Error	Database error, AI API unavailable
